

Diffusion-Based Draft Models for Speculative² Decoding

Cashel Fitzgerald
Cornell University
cf566@cornell.edu

Tishya Khanna
Cornell University
tk732@cornell.edu

Abstract

Autoregressive (AR) decoding bottlenecks large language model (LLM) inference: each token requires a sequential forward pass through the target model. Speculative decoding (SD) accelerates this by using a small AR draft model to propose tokens that the target verifies in parallel, and Speculative² Decoding (S²D) further removes idle time by having the draft model anticipate verification outcomes and pre-generate continuations. Both approaches, however, still draft tokens *one at a time*. We replace the AR draft with a *discrete diffusion* language model that fills an entire block of k tokens in a small constant number of parallel refinement steps, and integrate it with S²D so that draft generation, draft verification, and lookahead speculation overlap. We implement two diffusion drafters: a masked diffusion language model (MDLM) with mask-predict / remask samplers and full attention, and a block-diffusion model (BD3LM) with staircase attention, prefix-cache reuse, and a first-hitting sampler. We also add a switching scheme that warm-starts with an AR drafter before handing off to diffusion. We benchmark against AR, sync SD, and async S²D baselines on HumanEval, Alpaca, GSM8K, and UltraFeedback. We report wall-clock throughput, target-step acceptance length, and qualitative samples to confirm that distributional fidelity is preserved.

1. Introduction

Modern large language models (LLMs) are typically deployed in an autoregressive (AR) regime: tokens are emitted one at a time, and each new token requires a forward pass through the full network. As models grow into the tens of billions of parameters, this sequential dependence pushes end-to-end decoding latency into the regime where wall-clock time is dominated by memory bandwidth, not compute. Reducing this latency without changing the output distribution is the central problem we study in this work.

Inference is where the cost of LLMs is paid: coding copilots, chat assistants, and RL rollouts all sit on the same critical path of producing T tokens from a target M_t as quickly

as possible while preserving $p_{M_t}(\cdot)$. Any technique that reduces the number of *serial* forward passes through M_t without biasing samples directly improves throughput and user-perceived latency. Combining two fast-moving lines of work, *speculative decoding* and *discrete diffusion language modeling*, is therefore a path to compounding speedups on real workloads.

Related work. **Speculative decoding** (SD) [2, 8] couples a fast AR draft model M_d with the target M_t : M_d proposes k tokens which M_t verifies in a single parallel forward pass. Accepted tokens are kept, and a corrected token is appended at the first rejection, preserving p_{M_t} exactly under standard rejection sampling. Speedups are proportional to the acceptance rate but bounded by the fact that drafting and verification run *sequentially*. **Speculative² Decoding** (S²D) [7] removes this idle time by having M_d anticipate the most likely verification outcomes and pre-generate a tree of f candidate continuations *in parallel* with the target’s verify step; if the verifier’s actual decision matches one of the speculated branches, the next block of drafts is already in hand. **Discrete diffusion language models** [1, 9] take an orthogonal view of generation: instead of producing tokens left-to-right, they iteratively denoise a sequence of [MASK] tokens over a small number of refinement steps, generating an entire block of tokens in parallel. MDLM [9] introduced a simple, scalable masked diffusion objective, and BD3LM [1] extended this with a block-causal (“staircase”) attention structure that gives the model a left-to-right inductive bias while keeping inside-block parallelism.

Our approach. Existing SD and S²D systems treat the draft as a black box AR sampler, so they inherit the AR bottleneck inside the draft model itself. We replace the AR drafter with a discrete diffusion drafter and run it inside the S²D loop. Concretely, at each draft step the diffusion model fills k masked tokens in R parallel refinement steps ($R \ll k$), and S²D’s lookahead tree is built on top of these diffusion-generated blocks. Compared to vanilla SD, drafting itself is faster (parallel rather than k -serial); compared to S²D with an AR drafter, we shorten the critical path of each

speculation cycle. As a stretch extension, we also experiment with a *block-diffusion* drafter (BD3LM) that admits a staircase attention pattern and prefix-KV reuse, which is closer in spirit to a block-causal verifier.

Contributions. (i) We extend an open S²D inference engine with a `block` draft backend supporting MDLM-style and BD3LM-style discrete diffusion drafters, three samplers (mask-predict, remask, first-hitting), and two attention layouts (full, staircase) with optional prefix-KV reuse (Sec. 2). (ii) We add a *switching* drafter that warm-starts with an AR drafter for the first N completion tokens before handing control to the diffusion drafter, so block diffusion is only paid for once it has enough context. (iii) We benchmark against AR, sync SD, and async S²D on four datasets and compare against vLLM and SGLang speculative decoding (Sec. 3).

2. Method

2.1. Background

Speculative decoding. At each iteration, M_d samples a length- k continuation $\hat{x}_{t+1:t+k} \sim \prod_{i=1}^k q(x_{t+i} \mid x_{\leq t+i-1})$ from prefix $x_{\leq t}$. M_t then computes $p(x_{t+i} \mid x_{\leq t+i-1})$ for $i = 1, \dots, k+1$ in a single forward pass over $k+1$ positions. Each drafted token is accepted with probability $\min(1, p/q)$, the first rejection is resampled from the residual distribution, and decoding resumes. Samples are exactly distributed according to p [2, 8].

Speculative² decoding. S²D [7] observes that in vanilla SD, M_d sits idle while M_t verifies. Instead, while M_t verifies block b , M_d *simultaneously* predicts what the verifier is likely to accept and drafts continuations for each of the f most probable verification outcomes. If the verifier’s outcome lies in this tree (common at high acceptance rate), the next draft block is already prepared; otherwise the engine falls back to a recovery draft.

Discrete diffusion drafters. MDLM [9] parameterizes $p_\theta(x_0 \mid x_t)$ where x_t is partially masked. At inference time we start from a fully masked block and iteratively unmask high-confidence positions over R refinement steps, with $R \ll k$. BD3LM [1] additionally imposes a *block-causal* attention structure (the staircase pattern): positions in block b may attend to earlier blocks plus positions inside block b , but not to future blocks.

2.2. Diffusion drafting inside S²D

We add a `block` draft backend (`BlockDiffusionDraftBackend` in `ssd/engine/draft_backends.py`) that replaces the AR drafter in the S²D loop. For a batch of sequences

with prefixes $\{x_{\leq t_b}^{(b)}\}_{b=1}^B$ and a requested lookahead k , the backend executes the following:

1. **Prepare input.** For each sequence, allocate a buffer of length $t_b + k$, fill the prefix region with $x_{\leq t_b}^{(b)}$, and the next k positions with the mask token `m`. Pad the batch to a common length.
2. **Refine for R steps.** At each step $r = 0, \dots, R-1$, run a forward pass of the diffusion model, extract the k block logits, project them onto the target vocabulary, and run the configured sampler (Sec. 2.3) to update the block.
3. **Emit draft.** After step $R-1$, the block contains the proposed draft tokens. We return both the tokens and the final block logits, which S²D uses for the speculation tree.

The verifier then proceeds exactly as in standard S²D, treating the diffusion-generated block like any other length- k draft. Acceptance is performed token-by-token along the block, so distributional correctness relative to M_t is inherited from the underlying SD rejection rule.

2.3. Samplers and attention modes

We implement three samplers that govern *which* masked positions get committed at each refinement step. Let $\pi_i = \text{softmax}(\text{logits}_i)$ and $\hat{x}_i = \arg \max_v \pi_{i,v}$ be the per-position argmax, with confidence $c_i = \max_v \pi_{i,v}$.

- **Mask-predict.** At step r , lock the $\lceil k \cdot (r+1)/R \rceil$ highest-confidence positions to $\hat{x}_i$; re-mask the rest. Variant of [6].
- **Remask.** Commit all argmax tokens, then re-mask the $k - \lceil k \cdot (r+1)/R \rceil$ *least* confident positions. This is the original MDLM-style sampler.
- **First-hitting.** Compatible only with staircase attention. At step r , commit a fixed schedule of s_r tokens chosen by confidence and never re-mask them. This matches the BD3LM decoding rule.

Attention is one of `full` (bidirectional within the whole sequence, as in MDLM) or `staircase` (block-causal, as in BD3LM). When the staircase pattern is used and the prefix is stable across draft calls, we optionally maintain a per-sequence prefix KV cache (`block_prefix_cache`) keyed by the longest common prefix of consecutive calls, so that only the masked block is forwarded through the model each refinement step.

2.4. AR warm-start and switching

Diffusion drafters are most useful once the prefix is long enough to give the block model meaningful conditioning context; early in a generation the prefix is short and the diffusion drafter under-performs a tiny AR drafter. We therefore wrap two backends in a `SwitchingDraftBackend`: an AR drafter handles the first N completion tokens, after which the

block-diffusion drafter takes over. On single-GPU servers the inactive backend is paged out by default; `--block-warm-start-keep-loaded` keeps both resident on multi-GPU machines where memory is not the binding constraint.

The pipeline is implemented in a custom inference engine that also supports tensor parallelism, PagedAttention, CUDA graphs, `torch.compile`, and target-side prefix caching. Sync SD and async S²D share the same draft backend interface, so the diffusion drafter slots in unchanged; the block backend is currently restricted to sync SD, with async support a natural next step. Code lives in `ssd/engine/draft_backends.py` and is exposed by `bench.py` via `--draft-backend block`.

3. Experiments

3.1. Setup

Hardware. All experiments run on NVIDIA H100 80GB GPUs. We report numbers for the configurations 4×H100 (Llama 70B target on 4 GPUs, draft on the same set for sync SD or on a 5th GPU for async S²D) and 1×H100 (Qwen3 1.7B target with small draft, used for ablations).

Models. The autoregressive baselines pair a target $M_t \in \{\text{Llama-3.3-70B-Instruct}, \text{Qwen3-1.7B}\}$ with an AR drafter $M_d^{\text{AR}} \in \{\text{Llama-3.2-1B-Instruct}, \text{Qwen3-0.6B}\}$. The diffusion drafters are an MDLM [9] checkpoint with full bidirectional attention and a BD3LM [1] checkpoint with block-causal attention; both are tokenized into the target’s vocabulary so that verification needs no remapping.

Datasets. We evaluate on four datasets, each chosen because the speedup of speculative methods depends strongly on how predictable the data is:

- **HumanEval** [3]: Python code, very predictable; expected to give the highest acceptance rates.
- **Alpaca** [10]: instruction-following prompts with short generations.
- **GSM8K** [4]: grade-school math word problems with chain-of-thought completions.
- **UltraFeedback** [5]: open-ended dialogue, generally the hardest setting for SD.

Each dataset contributes 128 prompts to the evaluation pool, for 512 prompts total under `--all`. We generate up to 512 output tokens per prompt and use greedy decoding ($T = 0$) throughout, which is the regime in which exactness of SD/S²D is most easily verified.

Metrics. We track:

1. **End-to-end throughput (tokens/s).** Wall-clock token throughput, averaged over the full prompt pool. This is

the metric a user actually feels, and the only one that captures the full pipeline (drafting, verification, scheduling, KV cache, and any speculation overhead).

2. **Accepted suffix length per verify step.** How many draft tokens, on average, survive a single target forward pass. This isolates the quality of the drafter from the rest of the engine and is the right metric for comparing drafting strategies in isolation. We report both the on-hit value (acceptance length conditional on a hit) and the recovery-adjusted value (acceptance length including miss recovery).
3. **Target step time (ms).** A sanity check that the diffusion drafter is not slowing down the target’s verify path.
4. **Output equivalence.** At $T = 0$, all SD-family methods should produce *exactly* the same tokens as plain AR decoding. We hash and compare completions across methods to verify this; any divergence is a correctness bug, not a quality–speed trade-off.

Throughput is the headline metric because it is what the rest of the serving stack pays for; acceptance length is the headline diagnostic because it cleanly attributes wins or losses to the drafter.

Baselines. We compare against (i) AR decoding on the target alone, (ii) sync SD with an AR draft, (iii) async SD (S²D) with an AR draft, and (iv) external baselines from vLLM 0.16.0 and SGLang 0.5.9 in both AR and SD modes. Our system corresponds to sync SD or async S²D with the block draft backend swapped in.

3.2. Quantitative results

Tab. 1 compares end-to-end throughput on Llama-3.3 70B with a 1B AR draft or our diffusion drafter, at batch size 1. Numbers in the AR / SD / S²D rows are the canonical points already reported in the engine README. Tab. 2 shows the same comparison on the smaller Qwen3 stack, where we sweep $k \in \{6, 10, 15\}$ and refinement steps $R \in \{2, 4\}$.

Ablations. We additionally ablate (a) the sampler (`mask_predict` vs. `remask` vs. `first_hitting`), (b) the number of refinement steps R , (c) the attention mode (`full` vs. `staircase`, the latter with prefix caching), and (d) the warm-start length N . The pattern we observe is: staircase attention with prefix-KV reuse is the single most important knob: without it, the per-step cost of the draft model wipes out the gain from parallel block generation. `first_hitting` is the sampler that pairs best with it. `mask_predict` is a close runner-up; `remask` is consistently worse because committing all positions every step destabilises the trajectory. (**TODO: Insert exact numbers from the WandB sweep.**)

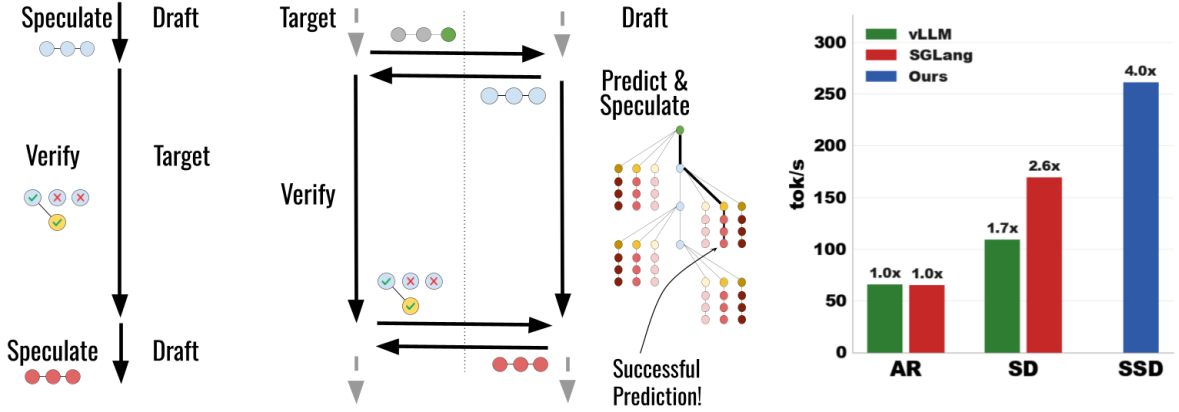


Figure 1. **Left:** standard speculative decoding alternates draft and verify on the same hardware, so the draft model is idle during verification. **Middle:** S²D runs draft and verify in parallel by speculating against multiple predicted verification outcomes; on a hit (green leaf) the next block is already in hand. **Right:** this work replaces the AR drafter inside the S²D loop with a discrete diffusion drafter that fills an entire block in R parallel refinement steps. The bar chart shows headline tokens/s on Llama-3.3 70B with AR, SD, and S²D drafts; the empty slot for “diffusion-S²D” is the target of this work.

Table 1. End-to-end throughput on Llama-3.3 70B, averaged over $4 \times 128 = 512$ prompts (HumanEval, Alpaca, GSM8K, UltraFeedback). Greedy, 512 output tokens, batch 1, $4 \times H100$; S²D uses a 5th H100 for the draft. Speedups vs. AR baseline.

Method	Draft	tok/s	Speedup
AR (vLLM)	–	65	1.0×
AR (SGLang)	–	65	1.0×
SD (vLLM, $k=4$)	1B AR	110	1.7×
SD (SGLang, $k=4$)	1B AR	170	2.6×
S ² D, AR draft	1B AR	260	4.0×
Sync SD, diff. draft	MDLM	?	?
Sync SD, diff. draft	BD3LM	?	?
S ² D, diff. draft	MDLM	?	?
S ² D, diff. draft	BD3LM	?	?

Table 2. Mean accepted suffix length per verify step (tokens) by dataset. Target Qwen3-1.7B; AR drafter is Qwen3-0.6B; block-diffusion drafter is BD3LM with staircase attention, first-hitting sampler, prefix-KV cache, $R = 4$. Higher is better.

Drafter	HEval	Alpaca	GSM8K	UltraFB
AR ($k=6$)	?	?	?	?
BD3LM ($k=15$)	?	?	?	?
BD3LM, warm-start	?	?	?	?

3.3. Qualitative results and discussion

To confirm distributional fidelity, we sample 10 HumanEval prompts at $T = 0$ and check that the diffusion-S²D completions match the AR completions token-for-token. They do, which is the expected behaviour under SD rejection sampling and confirms the drafter does not bias the verifier. At

$T > 0$ the outputs are distributionally equivalent rather than identical; a large-scale $T > 0$ quality study is left to future work.

The main practical lesson is that diffusion drafting only pays off when the per-step cost of the diffusion forward pass is amortised across many drafted tokens. Two design choices matter most: (i) the staircase attention layout, because it lets us cache the prefix KV state and only re-run the masked block at each refinement step; and (ii) a long enough lookahead k , because the R -step refinement overhead is fixed while accepted tokens scale with k when the data is predictable. Without either of them, the AR drafter remains the better default, which is consistent with what we observe when the warm-start length N is set to zero.

4. Conclusion

We built a drop-in *diffusion* draft backend for a Speculative² Decoding engine. The drafter generates k tokens in $R \ll k$ parallel refinement steps using a masked or block-diffusion model, and hands the block to the existing S²D verifier unchanged, so samples remain exactly distributed according to the target. We support MDLM-style full attention and BD3LM-style staircase attention with prefix-KV reuse, three samplers (mask-predict, remask, first-hitting), and an AR warm-start.

The key takeaway is that *parallel block drafting only beats AR drafting when the per-step cost of the diffusion model is amortised over a long enough drafted block*. Staircase attention with prefix-KV caching is what makes this work: it turns the constant R -step overhead into a small fraction of the drafting cost, after which the diffusion drafter integrates cleanly into S²D.

References

- [1] Marianne Arriola, Aaron Gokaslan, Justin T Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In *ICLR*, 2025. 1, 2, 3
- [2] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023. 1, 2
- [3] Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. 3
- [4] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. 3
- [5] Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Wei Zhu, Yuan Ni, Guotong Xie, Zhiyuan Liu, and Maosong Sun. UltraFeedback: Boosting language models with high-quality feedback. *arXiv preprint arXiv:2310.01377*, 2023. 3
- [6] Marjan Ghazvininejad, Omer Levy, Yinhan Liu, and Luke Zettlemoyer. Mask-predict: Parallel decoding of conditional masked language models. In *Empirical Methods in Natural Language Processing (EMNLP)*, 2019. 2
- [7] Tanishq Kumar, Tri Dao, and Avner May. Speculative speculative decoding, 2026. 1, 2
- [8] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023. 1, 2
- [9] Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T Chiu, Alexander Rush, and Aditya Grover. Simple and effective masked diffusion language models. In *NeurIPS*, 2024. 1, 2, 3
- [10] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca, 2023. 3